Arbre Binaire de Recherche (ABR)

1 Définition

Arbre binaire de recherche

Un ABR est un arbre binaire qui vérifie, de manière récursive :

- tous les nœuds du sous-arbre **gauche** ont une valeur **strictement inférieure** à celle de la racine ;
- tous les nœuds du sous-arbre **droit** ont une valeur **strictement supérieure** à celle de la racine ;
- les sous-arbres gauche et droit sont eux-mêmes des ABR.

Exemple et contre-exemple



*À gauche : **pas un ABR**. À droite : **un ABR**.*

Dans l'arbre de gauche, chaque nœud, pris avec ses deux fils, *semble* correct : $1 < 3 < 13$ et $2 < 8$. Pourtant 13, 21 et 2 (en violet foncé) violent la règle vis-à-vis de la racine 5 : 13 et 21 sont dans le sous-arbre gauche mais sont plus grands que 5 ; 2 est dans le sous-arbre droit mais est plus petit que 5.

Le parcours infixe d'un ABR

Le parcours **infixe** visite le sous-arbre gauche, puis la racine, puis le sous-arbre droit.

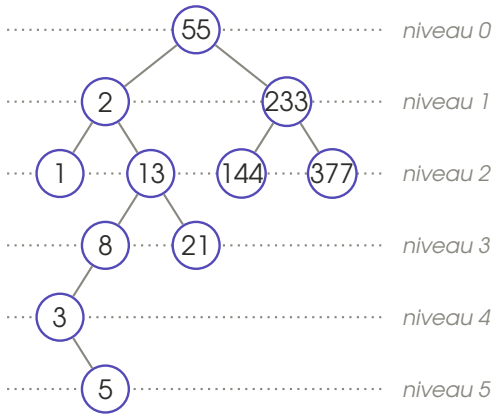
Propriété

Le parcours infixe d'un ABR produit les valeurs dans l'**ordre croissant**. Réciproquement, un arbre binaire dont le parcours infixe est **strictement croissant** est un ABR.

Pour l'ABR de droite, le parcours infixe donne $1, 2, 3, 5, 8, 13, 21, 34, 55$. Cette propriété fournit un test d'ABR (exercice 2) et un algorithme de tri (exercice 4).

2 Opérations sur les ABR

Toutes les figures de cette section utilisent l'ABR T ci-dessous (11 nœuds, hauteur 5). Son parcours infixe est $1, 2, 3, 5, 8, 13, 21, 55, 144, 233, 377$.

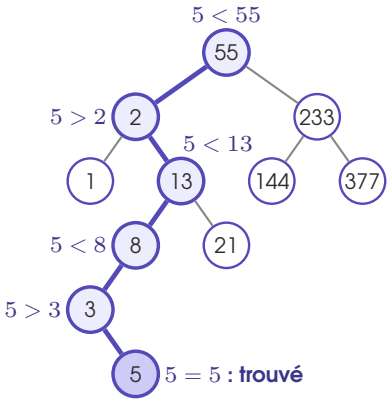


L'ABR T

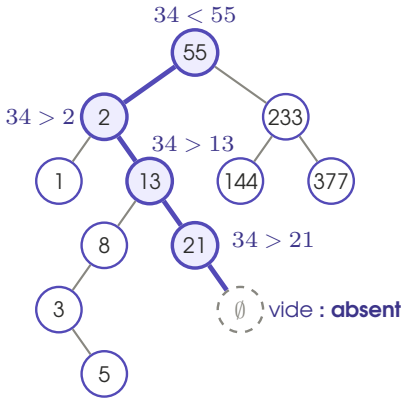
2.1 Recherche d'un élément

L'algorithme s'exécute récursivement. Pour chercher x dans l'arbre a :

- si a est **vide**, x est absent : on renvoie `False` ;
- si x est égal à la valeur de la racine, on renvoie `True` ;
- si x est plus petit, on cherche dans le sous-arbre **gauche** ; s'il est plus grand, dans le sous-arbre **droit**.



recherche de 5



recherche de 34

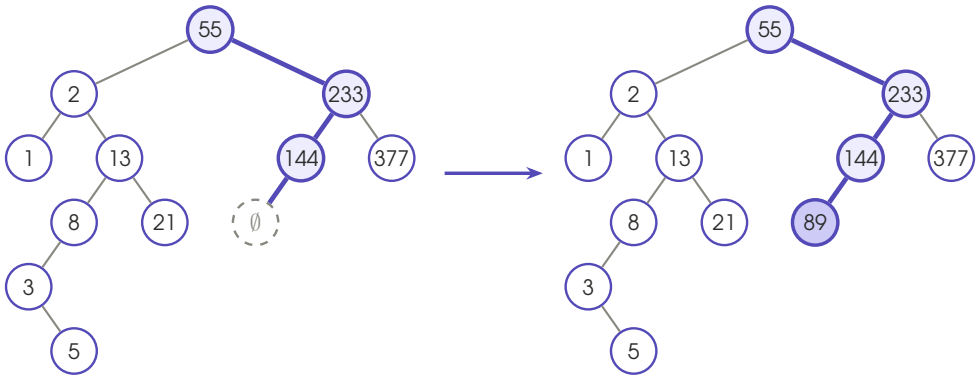
À RETENIR

À chaque comparaison, on **élimine un sous-arbre entier**. On visite au plus un nœud par niveau : au maximum $h + 1$ nœuds, quelle que soit la taille de l'arbre.

2.2 Insertion d'un élément

Pour insérer x dans l'arbre a :

- si a est **vide**, on crée un arbre dont la racine est x et dont les deux sous-arbres sont vides ;
- si x est plus petit que la racine, on insère dans le sous-arbre **gauche** ; s'il est plus grand, dans le sous-arbre **droit** ;
- si x est déjà présent, l'arbre ne change pas (valeurs distinctes).

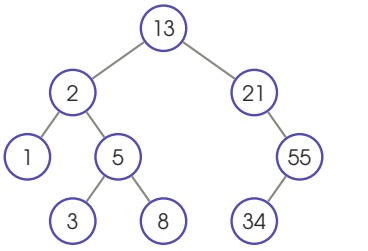


on descend comme pour une recherche de 89

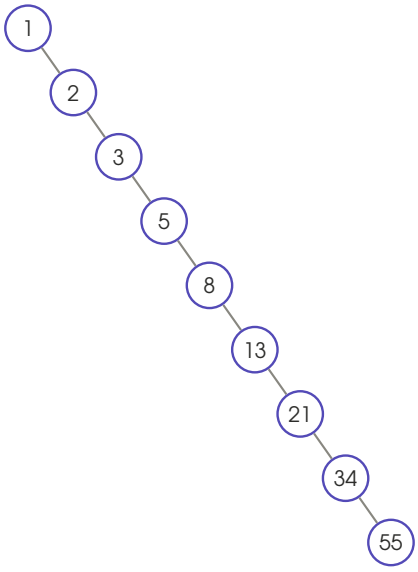
89 devient une feuille

> **À RETENIR**
> Une insertion se fait **toujours en feuille** : on suit exactement le chemin de la recherche, jusqu'à la première place vide. Aucun nœud existant n'est déplacé. La version fonctionnelle reconstruit seulement les nœuds du chemin racine → feuille.

L'ordre d'insertion décide de la forme. Les mêmes neuf valeurs donnent des arbres très différents selon l'ordre dans lequel on les insère :



ordre 13, 2, 21, 1, 5, 55, 3, 8, 34 : $h = 3$



ordre croissant 1, 2, 3, 5, 8, 13, 21, 34, 55 : $h = 8$

Insérer des valeurs déjà triées produit un arbre « en peigne » (dégénéré), qui ressemble à une liste : la recherche n'est alors pas plus rapide qu'un parcours séquentiel.

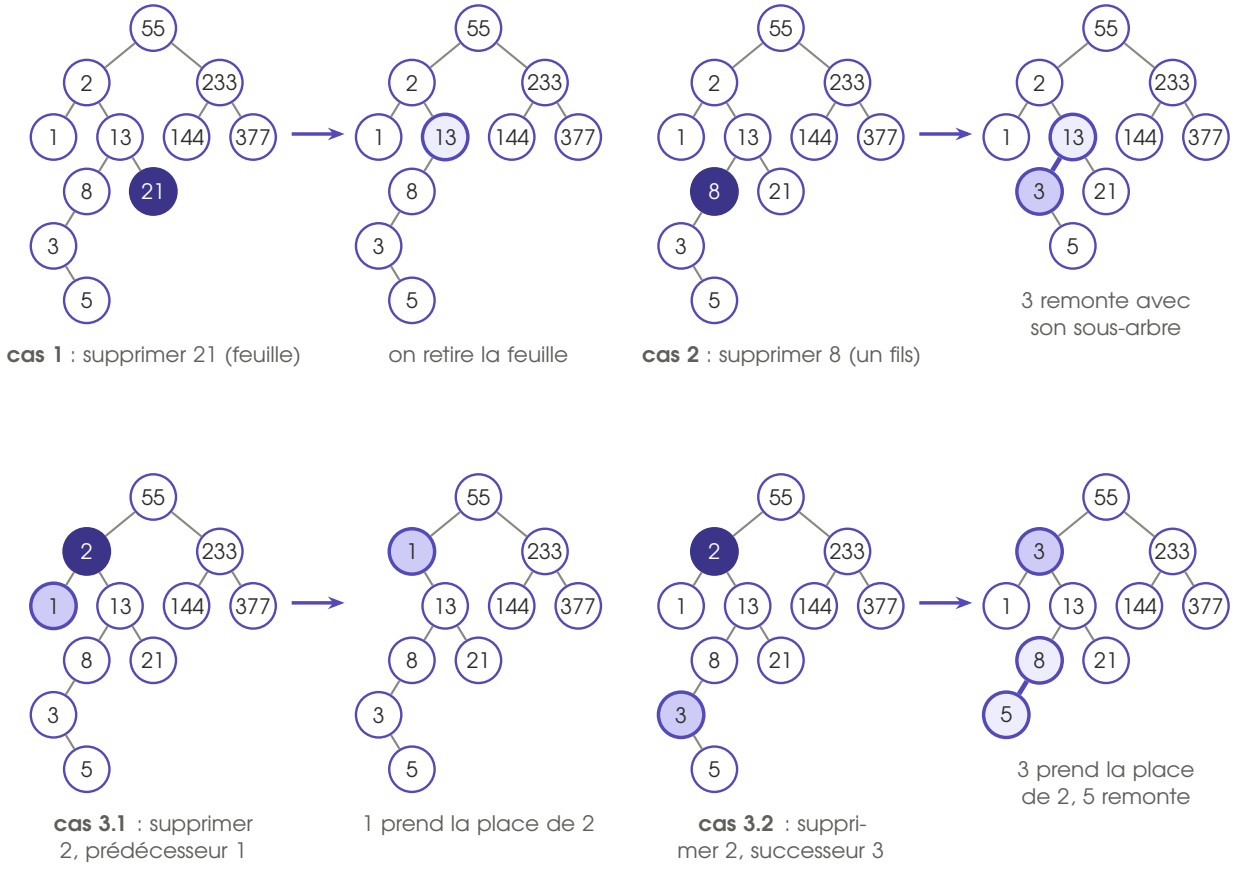
2.3 Suppression d'un élément

Pour supprimer le nœud contenant x (après l'avoir trouvé par recherche), trois cas :

1. **le nœud est une feuille** : on le retire ;
2. **le nœud a un seul fils** : on le remplace par ce fils (le fils « remonte » avec tout son sous-arbre) ;
3. **le nœud a deux fils** : on le remplace par une valeur qui préserve l'ordre, soit

- le **prédécesseur** : le maximum du sous-arbre gauche, soit
- le **successeur** : le minimum du sous-arbre droit.

Puis on supprime, dans le sous-arbre concerné, le nœud qui vient de monter.



Les trois cas, sur T. En violet foncé : le nœud supprimé ; en violet clair : ce qui change.

2.4 Complexité

Théorème

La recherche, l'insertion et la suppression dans un ABR de hauteur h s'exécutent en $\mathcal{O}(h)$.

Idée de la preuve. Chaque appel récursif descend d'un niveau et effectue un travail constant ; il y a donc au plus $h+1$ appels. Pour la suppression avec deux fils, la recherche du successeur prolonge la descente vers le bas : le chemin total reste de longueur au plus h.

Ce théorème ne dit rien de la taille n : tout dépend de la **forme** de l'arbre. Pour n nœuds,

$$\lceil \log_2(n+1) \rceil - 1 \;\leq\; h \;\leq\; n-1.$$

La borne basse est atteinte par un arbre « plein » ; la borne haute par l'arbre « en peigne ».

Tout l'enjeu est de **garder h petite**.

3 Exercices

Exercice 1 ● ○ ○ Arbres binaires quelconques

Ici l'arbre n'est *pas* ordonné : aucune propriété d'ABR ne peut être utilisée.

1. Écrire une fonction `max_arbre(arbre)` qui renvoie la valeur maximale d'un arbre binaire non vide.
2. Écrire une fonction `recherche_arbre(arbre, x)` qui indique si x appartient ou non à un arbre binaire.
3. Quel est le coût de `max_arbre` et de `recherche_arbre` dans le pire cas ?

Exercice 2 ● ● ○ ABR

1. Écrire une fonction `est_abr(arbre)` qui vérifie qu'un arbre binaire est un ABR.
2. Écrire une fonction `max_abr(arbre)` qui renvoie la valeur maximale d'un ABR non vide.
3. Écrire une fonction récursive, et une autre itérative (boucle `while`), `recherche_abr(arbre, x)` qui indique si x appartient ou non à un ABR.
4. Quel est le coût dans le pire cas de `max_abr`net de `recherche_abr` ? Comparer avec l'exercice 1.

Indication

Pour la question 1, une fonction qui compare seulement chaque nœud à ses deux fils échoue sur l'arbre de gauche de la figure de la section 1. Deux méthodes correctes : (1) vérifier que le parcours infixe est strictement croissant ; (2) faire descendre dans la récursion un intervalle $]bas, haut[$ dans lequel doit se trouver chaque valeur.

Exercice 3 ● ● ○ Insertion et suppression

1. Écrire une fonction récursive `insertion(abr, v)` qui insère `v` dans l'ABR `abr`. La fonction renvoie un nouvel arbre contenant la valeur insérée.
2. Écrire une fonction récursive `suppression(abr, v)` qui supprime `v` de l'ABR `abr`. La fonction renvoie l'arbre modifié.

Indication

Pour la suppression, écrire d'abord une fonction `minimum(abr)` ; traiter ensuite séparément : valeur différente de la racine (descente), puis les cas 1, 2, 3.

Exercice 4 ● ● ● Aller plus loin : construire et trier

1. Écrire `construire(valeurs)` qui renvoie l'ABR obtenu en insérant successivement les valeurs d'une liste dans un arbre initialement vide.
2. En déduire `tri_abr(liste)` qui trie une liste de valeurs distinctes en utilisant le parcours infixe.
3. Donner la complexité de `tri_abr` dans le meilleur et dans le pire cas.

4 Corrigés

Exercice 1

```
def max_arbre(a):                       # arbre non vide, non ordonné
    m = a[0]
    if a[1] != []:
        m = max(m, max_arbre(a[1]))
    if a[2] != []:
        m = max(m, max_arbre(a[2]))
    return m

def recherche_arbre(a, x):
    if a == []:
        return False
    return a[0] == x or recherche_arbre(a[1], x) or recherche_arbre(a[2], x)
```

Question 3. Comme rien n'est ordonné, `max_arbre` doit visiter tous les nœuds, et `recherche_arbre` peut devoir le faire aussi (élément absent ou dans la dernière feuille visitée) : les deux sont en $\mathcal{O}(n)$.

Exercice 2

Question 1, méthode (1) : parcours infixe.

```
def infixe(a):
    if a == []:
        return []
    return infixe(a[1]) + [a[0]] + infixe(a[2])

def est_abr_infixe(a):                  # valeurs distinctes
    l = infixe(a)
    for i in range(len(l) - 1):
        if l[i] > l[i + 1]:
            return False
    return True
```

Question 1, méthode (2) : bornes. Chaque valeur doit se trouver dans un intervalle ouvert dont les bornes viennent des ancêtres : en descendant à gauche, la borne haute devient la valeur du nœud ; en descendant à droite, c'est la borne basse.

```
def est_abr(a, bas=float('-inf'), haut=float('inf')):
    if a == []:
        return True
    if not (bas < a[0] < haut):       # a[0] doit être dans l'intervalle ]bas, haut[
        return False
    return est_abr(a[1], bas, a[0]) and est_abr(a[2], a[0], haut)
```

La méthode (2) est en $\mathcal{O}(n)$, comme (1), mais sans construire de liste.

Questions 2 et 3.

```
def max_abr(a):                          # arbre non vide : tout à droite
    if a[2] == []:
        return a[0]
    return max_abr(a[2])

def recherche_abr_r(a, x):           # version récursive
    if a == []:
        return False                 # arbre vide : x est absent
    if x == a[0]:
        return True
    if x < a[0]:
        return recherche_abr_r(a[1], x)      # plus petit : à gauche
    return recherche_abr_r(a[2], x)          # plus grand : à droite

def recherche_abr_i(a, x):           # version itérative
    while a != []:
        v, g, d = a
        if x == v:
            return True
        a = g if x < v else d
    return False
```

Question 4. `max_abr` descend toujours à droite, sans comparaison, jusqu'au dernier nœud : au plus $h + 1$ appels. Dans `recherche_abr`, chaque comparaison élimine un sous-arbre, donc au plus un nœud par niveau est visité. Les deux sont en $\mathcal{O}(h)$, alors que `max_arbre` et `recherche_arbre` sont en $\mathcal{O}(n)$: c'est tout l'intérêt de la structure.

Exercice 3

```
def insertion(a, x):
    if a == []:
        return [x, [], []]              # place vide : nouvelle feuille
    if x < a[0]:
        return [a[0], insertion(a[1], x), a[2]]
    if x > a[0]:
        return [a[0], a[1], insertion(a[2], x)]
    return a                            # déjà présent : rien à faire

def minimum(a):
    if a[1] == []:
        return a[0]
    return minimum(a[1])

def suppression(a, x):
    if a == []:
        return []                       # x absent
    if x < a[0]:
        return [a[0], suppression(a[1], x), a[2]]
    if x > a[0]:
        return [a[0], a[1], suppression(a[2], x)]
    # x == a[0] : c'est le nœud à supprimer
```

```
    if a[1] == []:                          # pas de fils gauche
        return a[2]
    if a[2] == []:                          # pas de fils droit
        return a[1]
    m = minimum(a[2])                       # deux fils : on prend le successeur
    return [m, a[1], suppression(a[2], m)]
```

Exercice 4

```
def construire(valeurs):
    a = []
    for x in valeurs:
        a = insertion(a, x)
    return a

def tri_abr(liste):
    return infixe(construire(liste))
```

Question 3. Meilleur cas (arbre équilibré) : n insertions de coût $\mathcal{O}(\log n)$, soit $\mathcal{O}(n \log n)$. Pire cas (liste déjà triée, arbre en peigne) : $1 + 2 + \cdots + n$ comparaisons, soit $\mathcal{O}(n^2)$. Dans les deux cas, le parcours infixe coûte $\mathcal{O}(n)$.